Historical Weather Pattern Matching via NLP Analysis of Forecast Discussions

Leif Rogers (lfr42) Jingyu Xu (jx62) Eric Gao (xg255)

May 2026

Presentation recording: `https://drive.google.com/file/d/1Ao-2w7cCg69h0Zl_Mj87jfvxlnU-BalM/view?usp=sharing`
AI Keywords: Natural Language Processing, Information Extraction, Semantic Similarity, Transformer Models
Application setting: Weather forecasting and meteorological analysis
Other contributors: N/A

1 Project Description

1.1 What we planned

NWS forecasters write Area Forecast Discussions (AFDs) several times a day, describing current atmospheric conditions and what they expect to happen. These narratives are detailed and information-rich, but because they are unstructured text, there is no easy way to search across years of them and ask "when have we seen conditions like this before?"

We wanted to build an NLP system that takes a current AFD and retrieves the most semantically similar historical discussions from a large archive, then shows what weather actually occurred on those dates. To make the retrieval meaningful, we planned to train text embeddings using observed weather data from NOAA archives as a training signal. By pushing together AFDs that preceded similar actual weather outcomes, the model would learn to capture underlying meteorological meaning rather than just surface-level wording. This was intended to let us build a retrieval system without manually labeling thousands of discussions by event type.

The central AI challenge is learning text representations of forecast discussions that capture meteorological meaning rather than just lexical overlap. A naive keyword search would miss the fact that "persistent northwesterly flow off Cayuga Lake" and "lake-enhanced snow bands setting up in the Ithaca area" describe closely related setups. Our primary approach was to fine-tune a pretrained transformer using contrastive learning: pair each AFD with its corresponding observed weather, define a similarity function over the weather observations, and train so that AFDs whose outcomes were similar get pulled together in embedding space, while those with different outcomes get pushed apart.

As a comparison we also planned a more explicit approach: extract structured event frames from each AFD with fields like event type, intensity, region, timing, and mechanism (either with BIO sequence labeling on RoBERTa, or with few-shot LLM prompting). Retrieval would then become matching over those structured fields instead of comparing dense vectors. Running both approaches lets us see whether learned embeddings or explicit extraction does a better job.

1.2 What the project actually became

We built and evaluated four retrieval methods against held-out test AFDs:

1. A random baseline (floor),
2. Zero-shot retrieval using a pretrained Jina v2 sentence embedding,
3. A contrastive fine-tune of the same embedding using triplet loss with hardest-positive / hardest-negative mining over a 37-dimensional weather vector,
4. A structured-extraction retrieval that parses each AFD into JSON event frames via a large language model and matches by frame-field overlap.

Data covers 2007–2025 for the NWS Binghamton (BGM) county warning area, with per-station METAR observations from KBGM, KRME, KAVP, KELM, KITH plus CWA-wide snow totals from NOAA LCD, paired with the daily AFD corpus from IEM. After cleaning we had 6,810 AFD–weather pairs, split 80/10/10 into train/val/test.

1.3 Key aspects of AI

The core AI content is in two places:

Contrastive sentence representation learning. We fine-tune `jinaai/jina-embeddings-v2-base-en` using a triplet loss $\mathcal{L} = \max\big(0,\, d(a,p) - d(a,n) + m\big)$ where the positive and negative for each anchor AFD are mined by full-pairwise search over a similarity defined on observed weather. The weather similarity itself is the normalized Euclidean distance over a 37-dimensional vector built from per-station temperature, wind, gust, precipitation, thunderstorm and freezing-rain indicators, plus CWA-wide snow totals. This makes the textual representation learn to pull together AFDs that preceded similar real-world outcomes, rather than those that share surface vocabulary.

LLM-based structured information extraction. As a comparison approach the proposal called for either BIO sequence labeling with RoBERTa or LLM prompting. We chose LLM prompting because the project has no annotated training data for BIO. Each AFD is sent to Anthropic's `claude-sonnet-4-6` (via the Claude Code CLI in print mode) with a few-shot prompt asking for a JSON array of event frames $\{\textit{event\_type}, \textit{intensity}, \textit{region}, \textit{time}, \textit{mechanism}\}$. The event-type field is constrained to a small closed vocabulary (snow, rain, freezing rain, sleet, thunderstorm, wind, fog, heat, cold, lake-effect, other) and the intensity field similarly. Retrieval is then a Jaccard overlap on the frame signatures. Each AFD is reduced to a set of tokens, one for each event type mentioned, one for each (*event*, *intensity*) pair, and one for the first word of each mechanism string. Two AFDs are scored by the Jaccard index of their token sets, $J(A,B) = |A \cap B| / |A \cup B|$, which is the fraction of tokens shared between the two sets relative to the total distinct tokens across both. A score of 1 means the two AFDs have an identical set of described events; 0 means no shared events at all. Jaccard ignores token counts, so an AFD that mentions snow ten times scores the same against another snow-mentioning AFD as one that mentions snow once. That suits us, since the question is not how much was said about snow but whether snow was described at all.

1.4 Data pipeline

AFD text comes from the Iowa Environmental Mesonet's NWS text archive. For each calendar date in the BGM forecast area we keep the single longest issuance (multiple AFDs are issued per day; the longest is usually the main discussion rather than a short update). Hourly METAR observations come from the IEM ASOS API for KBGM, KRME, KAVP, KELM, and KITH, and are aggregated to daily summaries: max and min temperature, mean wind, max gust, total precipitation, plus a binary thunderstorm flag (any `TS`-containing weather code) and a binary freezing-rain flag (any `FZRA` code). Daily snow totals and snow depth come from NOAA NCEI Local Climatological Data at KBGM. The final 37-dimensional similarity vector is built by concatenating the per-station fields for the five stations with the two CWA-wide snow fields.

We dropped records prior to 2007 because KRME did not report and LCD snow data was not yet available, leaving an inhomogeneous feature space across the corpus. Sensor outliers were filtered: max gust readings above 100 mph (a regional ceiling) were dropped as bad data, which removed 16 records mostly clustered at KAVP in 2003–2007. After alignment we had 6,810 AFD–weather pairs, randomly split 80/10/10 into train/val/test using a fixed seed. The 681-record test split is written to disk before training begins and is never seen by any model.

1.5 Scope changes from the proposal

We dropped the qualitative user study from the original proposal (recruit atmospheric science students for a blind rating of analog quality). The change was made late in the semester for time reasons; the quantitative evaluation in Section 2 stands on its own. We also pivoted on the LLM-extraction comparison: the proposal listed BIO sequence labeling on RoBERTa as the leading option, but we used LLM prompting instead because no annotated training data existed and prompting was both faster to implement and easier to iterate on.

2 Evaluation

2.1 Questions asked

1. Does contrastive fine-tuning on the weather-similarity signal produce better retrievals than a strong pretrained embedding?
2. Does an explicit structured-extraction approach beat dense embedding retrieval?
3. How far above random does any of this get us?

2.2 Methodology

For each AFD in the held-out test set ($n = 681$) we retrieve the top-K most similar AFDs from the rest of the corpus ($n = 6{,}129$) and compute the mean weather similarity between the test AFD's actual observed weather and the actual weather of each retrieved match. The metric is the mean across the test set; higher is better. We report $K \in \{1, 3, 5, 10\}$, standard deviation across queries, a 95% confidence interval on the mean, and a paired t-statistic for fine-tuned versus zero-shot. The random baseline is averaged over 20 independent draws to stabilize the floor.

The weather similarity function used both during training (to mine triplets) and during evaluation (to score retrievals) is the same: $1 - \|v_a - v_b\|/\text{MAX\_DIST}$ on the 37-dimensional normalized vector.

Note on the metric's range. MAX_DIST is the theoretical maximum L2 distance assuming every feature is simultaneously at its climatological extreme (e.g., temperatures at -25°F and +95°F at the same time). Real pairs of days never get close to that bound, so observed similarity values cluster well above 0.5. On top of this compression, all our records come from a single CWA in upstate New York, where any two random January days share a winter climatology and look broadly similar. Both factors push the random baseline up to roughly 0.79.

Because the absolute scale is misleading, we also report a *skill score* that re-bases the comparison against random: $\text{skill}(m) = \big(m - \text{random}\big)/\big(1 - \text{random}\big)$. Random scores 0% by definition; a perfect retriever scores 100%.

How to read the paired comparison numbers. When we compare the fine-tuned model against zero-shot we use a paired test: for each of the 681 test queries we have both models' scores on that same query, so we can compute a per-query difference. We report two things from those 681 differences. The paired t-statistic is the mean difference divided by its standard error; values with $|t| > 2$ are conventionally treated as statistically significant, and $|t| > 3.29$ corresponds to $p < 0.001$. The win rate is the fraction of queries where the fine-tuned model beat zero-shot; 50% would be a coin flip, anything below means it loses on the majority of queries.

2.3 Results

Method	K=1	K=3	K=5	K=10
Random	0.7904 ± 0.006	0.7935 ± 0.004	0.7939 ± 0.004	0.7929 ± 0.003
Zero-shot Jina	0.8469 ± 0.004	0.8415 ± 0.003	0.8389 ± 0.003	0.8358 ± 0.003
Contrastive FT	0.8381 ± 0.005	0.8361 ± 0.003	0.8339 ± 0.003	0.8319 ± 0.003
Extraction (Jaccard)	**0.8495** ± 0.004	**0.8442** ± 0.003	**0.8425** ± 0.003	**0.8412** ± 0.003

Table 1: Mean top-K weather similarity on the held-out test set, with 95% CI half-width. Higher is better. Random is averaged over 20 seeds.

Method	K=1	K=3	K=5	K=10
Random	0%	0%	0%	0%
Zero-shot Jina	27.0%	23.2%	21.8%	20.7%
Contrastive FT	22.8%	20.6%	19.4%	18.8%
Extraction (Jaccard)	**28.2%**	**24.6%**	**23.6%**	**23.3%**

Table 2: Skill score $(m - \text{random})/(1 - \text{random})$ at each K. 0% means tied with random, 100% means perfect retrieval. The structured-extraction approach is the strongest at every K, and the contrastive fine-tune is the weakest of the three learned methods.

Contrastive fine-tuning hurt retrieval. Across every K the fine-tuned model was statistically significantly *worse* than the zero-shot baseline. At K=1 the mean difference was -0.0088 with paired $t = -3.47$ over 681 queries ($p < 0.001$), and the fine-tuned model beat the zero-shot model on only 35.8% of queries. The same pattern held at K=3, 5, 10 with $t \in [-3.85, -3.29]$ and per-query win rates between 44.5% and 46.4%.

Pretrained embeddings already capture meaningful weather semantics. The zero-shot model lifted retrieval similarity 5.6 percentage points above random at K=1 (0.7904 $\rightarrow$ 0.8469). That is the real win of the project: the textual content of an AFD, embedded by a generic pretrained sentence transformer, is predictive of the actual weather that followed without any task-specific training.

Structured extraction was the strongest method. Jaccard retrieval over LLM-extracted event frames beat zero-shot dense retrieval at every K (e.g., 0.8495 vs. 0.8469 at K=1) and beat the contrastive fine-tune by a wider margin (0.8495 vs. 0.8381). In skill-score terms it closes 28.2% of the gap to perfect retrieval at K=1, versus 27.0% for zero-shot and 22.8% for the fine-tune. Each AFD was reduced to a bag of event-type, intensity, and mechanism tokens via a few-shot Claude prompt, and matched by set Jaccard. The lift is small in absolute terms but consistent across all four K values, and is achieved with no task-specific training.

2.4 Why the contrastive approach failed

We see three plausible reasons, in roughly decreasing order of likelihood.

1. **Training set is too small.** 6,129 anchors produce 6,129 triplets, which the model overfits in fewer than 100 gradient steps. Validation triplet ordering accuracy actually declined epoch-over-epoch (epoch 1: 85.76%, epoch 2: 84.73%, epoch 3: 83.26%). The best-by-evaluator checkpoint was epoch 1 and even that already underperforms the pretrained model on the downstream retrieval task.

2. **Hardest-pair mining is too aggressive.** The miner pairs each anchor with its single most-similar and most-dissimilar AFD over the entire training set. With a small corpus this means the model repeatedly sees the same extreme pairs and learns to memorize them rather than to generalize.

3. **Pretrained prior is strong, target signal is weak.** Jina v2 was trained on hundreds of millions of sentence pairs. Our downstream similarity is a normalized Euclidean distance on a 37-dimensional vector with a high seasonal autocorrelation floor (random retrieval already scores 0.79). The contrastive update damages the pretrained representation more than the weather signal can repair it.

2.5 Why structured extraction won

The lift is small in absolute terms (one to two percentage points of skill at K=1) but consistent across every K. Two things probably explain it.

The first is just model size. Jina v2 base has 137M parameters and was trained on generic sentence pairs. The LLM doing the extraction is much larger and fine-tuned to follow instructions. Mapping AFD prose into a small set of event-type labels plays to its strengths, and the resulting token set is a much tidier signal than a 768-dim dense vector from a smaller encoder trained for a different objective.

The second is what each representation chooses to keep. AFDs are full of forecaster reasoning, model agreement discussion, ensemble caveats, and other meta-commentary that doesn't predict the weather that actually verifies. The dense embedding has to encode all of that, since it has no way to know what to ignore. The frame representation throws it away and keeps only the events. For a retrieval task whose ground truth is observed weather, throwing the meta-content away helps.

The practical lesson is that on a small, specialized retrieval problem, an explicit pipeline that uses a strong LLM as a feature extractor can do better than fine-tuning a smaller encoder, especially when only a small part of the source text actually matters.

2.6 What we would do differently

The first thing we would do is scale up the corpus. 6,810 records from a single CWA is barely enough for contrastive learning to overcome a strong pretrained prior. Adding adjacent NWS offices in the northeast (BUF, ALB, CTP, OKX, etc.) would bring the corpus to 50,000–100,000 AFDs while keeping climate broadly comparable. CONUS-wide scrape would yield around a million.

Even before that, we would change three things about the training loop. First, refresh the hardest-pair miner between epochs rather than mining once and reusing for all three epochs; once the model fits a triplet it stops being informative. Second, try `MultipleNegativesRankingLoss` (in-batch negatives) in place of explicit triplet mining; this is the standard sentence-transformer recipe for retrieval fine-tuning and tends to be more stable on small data. Third, target day-after weather rather than same-day weather, since AFDs are forecasts of what is going to happen rather than descriptions of current conditions.

On the extraction side, the bag-of-tokens Jaccard score is the simplest thing that could possibly work and it still won. A richer scoring function, for example weighting matches on event type more than mechanism, or learning a small classifier over frame-pair features, would likely lift the result further. Combining extraction features with embedding features through a learned reranker is the obvious next step.

Finally, the qualitative user study from the proposal was dropped for time. With the quantitative numbers in hand it would now be straightforward to set up: sample 20–30 query AFDs, show each rater the top-3 retrievals from the strongest model, ask whether the retrieved analogs are meteorologically reasonable. That gives a human-grounded check on the quantitative skill scores.

2.7 Conclusion: did we succeed?

Each of the three questions from Section 2.1 got a clear quantitative answer:

- **Q1. Does contrastive fine-tuning beat the zero-shot baseline?** No. The fine-tuned model was significantly worse than zero-shot at every K (paired $t \in [-3.85, -3.29]$, $p < 0.001$; per-query win rate 35.8%–46.4%). The proposal's main hypothesis was not supported in this regime.
- **Q2. Does structured extraction beat dense retrieval?** Yes. Frame-Jaccard scored higher than both zero-shot and contrastive at every K, reaching 28.2% skill at K=1 versus 27.0% and 22.8%.
- **Q3. How far above random does any method get us?** Roughly 20–28% of the way from random to perfect retrieval, depending on K. All three learned methods are clearly above the random floor with non-overlapping 95% confidence intervals.

Overall we consider the project a partial success. The negative result on the main hypothesis is still a useful finding: we know quantitatively that contrastive fine-tuning on a corpus this size damages retrieval rather than helping it, and we have stated three concrete reasons we believe explain that result. The most useful positive finding for anyone building an AFD analog system is that a pretrained sentence transformer used zero-shot already lifts retrieval similarity well above

chance, and that explicit LLM-based extraction can lift it further without any task-specific training. The least useful is the contrastive fine-tune as we implemented it: more data, refreshed mining, and a different loss function would all be needed before this approach is viable.

References

- Jina AI. *jina-embeddings-v2-base-en.* `https://huggingface.co/jinaai/jina-embeddings-v2-base-en`
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT. EMNLP-IJCNLP.
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. CVPR.
- Iowa Environmental Mesonet, NWS Text Archives. `https://mesonet.agron.iastate.edu/`
- NOAA NCEI Local Climatological Data. `https://www.ncei.noaa.gov/cdo-web/`
- FAISS. `https://github.com/facebookresearch/faiss`

Software Resources

- Hugging Face Transformers and sentence-transformers libraries (model loading, triplet loss, evaluator)
- PyTorch (training)
- FAISS (approximate nearest neighbor over embeddings)
- Anthropic Claude — model `claude-sonnet-4-6`, accessed via the Claude Code CLI in print mode (LLM prompting for structured extraction)

Data Resources

- NWS AFD text archive via IEM (Iowa Environmental Mesonet), 2007–2025
- IEM ASOS hourly METAR observations for five BGM-CWA stations: KBGM, KRME, KAVP, KELM, KITH
- NOAA NCEI Local Climatological Data for daily snow totals and depth, station KBGM

Relationship to Other Work

N/A. This project is not related to any other work that any team member is currently involved in.

Use of AI Tools

Anthropic's Claude was used in two distinct roles in this project. First, as a core methodological component of the system itself: every AFD in the corpus was passed through `claude -p` with a few-shot prompt to extract structured event frames, which were then used for the frame-Jaccard retrieval method described above. Second, as an authoring assistant: we used Claude to help write portions of the codebase, debug pipeline issues during development, and draft sections of this report and the accompanying slides. All design decisions, experimental choices, results, and interpretations are our own.